

INSTITUTO FEDERAL
Fluminense

Campus
Bom Jesus do Itabapoana

Projeto e análise de algoritmos

Algoritmos de Força Bruta
Algoritmos de Divisão e Conquista
Algoritmos Gulosos

Professora Ana Mara de Oliveira Figueiredo
ana.figueiredo@iff.edu.br

Algoritmos de força bruta

Os algoritmos de força bruta são uma abordagem simples e direta para resolver problemas computacionais, onde todas as possíveis soluções são exploradas sistematicamente até encontrar a resposta correta. Apesar de sua simplicidade, eles podem ser eficazes para problemas menores ou quando não há uma solução algorítmica mais eficiente disponível.

Características dos Algoritmos de Força Bruta:

Exaustividade: Os algoritmos de força bruta consideram todas as combinações possíveis de entrada para encontrar a solução, sem pular etapas. Isso garante que nenhuma solução seja deixada de lado.

Simplicidade: A principal vantagem dessa abordagem é sua simplicidade conceitual. Ela envolve a aplicação direta das regras do problema para todas as entradas possíveis.

Garantia de Correção: Como todas as possíveis soluções são verificadas, os algoritmos de força bruta garantem que a resposta correta será encontrada, desde que todas as combinações sejam examinadas.

Exemplos de Algoritmos de Força Bruta:

Pesquisa Linear: Para encontrar um elemento em uma lista não ordenada, a pesquisa linear verifica cada elemento sequencialmente até encontrar o elemento desejado ou percorrer toda a lista.

Geração de Permutações e Combinações: Problemas de permutação (ordenação de elementos) e combinação (seleção de elementos) podem ser resolvidos gerando todas as possíveis permutações/combinações e avaliando-as.

Quebra de Senhas: Algoritmos de força bruta são usados para quebrar senhas testando todas as combinações possíveis até encontrar a senha correta.

Resolução de Quebra-Cabeças: Em jogos e quebra-cabeças, a força bruta pode ser usada para testar todas as sequências possíveis de movimentos até encontrar a solução.

Limitações dos Algoritmos de Força Bruta:

Ineficiência para Grandes Entradas: À medida que o tamanho da entrada aumenta, a abordagem de força bruta se torna cada vez mais lenta e ineficiente. Isso ocorre porque a quantidade de combinações possíveis cresce exponencialmente.

Recursos Computacionais: Algoritmos de força bruta podem ser intensivos em recursos computacionais, como tempo de processamento e memória, tornando-os inadequados para problemas complexos.

Falta de Elegância: A abordagem de força bruta muitas vezes não é a mais elegante em termos de design de algoritmo. Outras abordagens, como algoritmos de divisão e conquista ou programação dinâmica, podem ser mais eficientes.

Quando Usar Algoritmos de Força Bruta:

Problemas Pequenos: Algoritmos de força bruta são adequados para problemas pequenos, onde o número de combinações possíveis é gerenciável.

Verificação de Soluções: Essa abordagem é útil para verificar a corretude de algoritmos mais eficientes, pois garante uma resposta correta.

Soluções Inexistentes: Em alguns casos, não há uma solução algorítmica mais eficiente conhecida. A força bruta pode ser a única opção.

Algoritmos de Divisão e Conquista

Os algoritmos de divisão e conquista são uma estratégia de resolução de problemas que envolve a divisão de um problema maior em subproblemas menores, a resolução desses subproblemas e a combinação das soluções para obter a solução final. Esse paradigma é amplamente utilizado na ciência da computação para resolver problemas complexos de maneira eficiente. Aqui está um resumo detalhado dos algoritmos de divisão e conquista:

Princípios Fundamentais:

Dividir: O problema é dividido em subproblemas menores, geralmente do mesmo tipo do problema original.

Conquistar: Os subproblemas são resolvidos recursivamente. Quando os subproblemas são pequenos o suficiente, eles são resolvidos diretamente.

Combinar: As soluções dos subproblemas são combinadas para obter a solução do problema original.

Exemplos Clássicos:

- **Mergesort:**
 - Divide a lista não ordenada em duas metades.
 - Ordena as duas metades usando o mergesort recursivamente.
 - Combina as metades ordenadas usando a operação de "merge".
 - Complexidade de tempo: $O(n \log n)$ no pior caso.
- **Quicksort:**
 - Escolhe um elemento pivot e divide a lista em elementos menores e maiores que o pivot.
 - Ordena as sublistas menores e maiores usando o quicksort recursivamente.
 - Concatena as sublistas ordenadas com o pivot no meio.
 - Complexidade de tempo médio: $O(n \log n)$, pior caso: $O(n^2)$ (com pivot desbalanceado).
- **Multiplicação de Matrizes (Algoritmo de Strassen):**
 - Divide as matrizes em submatrizes menores.
 - Realiza multiplicações recursivas das submatrizes.
 - Combina as submatrizes multiplicadas para obter a matriz final.
 - Complexidade de tempo: $O(n^{\log_2 7}) \approx O(n^{2.81})$ em comparação com $O(n^3)$ no algoritmo ingênuo.

Vantagens:

Eficiência em Problemas Recorrentes: Problemas que podem ser divididos em subproblemas menores frequentemente têm algoritmos de divisão e conquista mais eficientes em termos de tempo do que abordagens ingênuas.

Utilização de Recursão: A recursão é uma ferramenta natural para implementar algoritmos de divisão e conquista, tornando-os mais fáceis de entender e implementar.

Exploração de Paralelismo: Muitos algoritmos de divisão e conquista podem ser paralelizados, explorando recursos de hardware modernos.

Desafios e Considerações:

Overhead da Recursão: A recursão pode resultar em sobrecarga devido às chamadas de função e à alocação de memória.

Escolha de Subproblemas: A escolha adequada de como dividir o problema é crucial para obter um algoritmo **eficiente**.

Caso Base Correto: Definir um caso base correto para a recursão é importante para evitar loops infinitos.

Algoritmos Gulosos

Os algoritmos gulosos são uma estratégia de resolução de problemas que busca soluções ótimas localmente em cada etapa, na esperança de alcançar uma solução global ótima. Essa abordagem segue a ideia de fazer escolhas que parecem as melhores naquele momento, sem considerar o impacto total futuro.

Estratégia Gulosa:

- Os algoritmos gulosos funcionam fazendo uma série de escolhas locais que, em teoria, levariam a uma solução global ótima.
- A estratégia é eficaz quando as escolhas locais não afetam negativamente a solução global.

Exemplos:

Problema da Escolha da Moeda: Um exemplo clássico é o problema de trocar uma quantia usando o menor número de moedas possíveis. O algoritmo guloso escolheria a maior moeda possível em cada etapa, visando reduzir o número total de moedas.

Problema da Mochila Fracionária: Dado um conjunto de itens com valores e pesos, o objetivo é encher uma mochila com a maior quantidade de valor possível, respeitando a capacidade da mochila. O algoritmo guloso seleciona itens com a maior relação valor/peso primeiro, preenchendo a mochila de forma fracionária se necessário.

Problema de Escalonamento de Atividades: Dado um conjunto de atividades com tempos de início e término, o objetivo é selecionar o maior número possível de atividades não sobrepostas. O algoritmo guloso seleciona a próxima atividade que termina mais cedo, liberando tempo para atividades futuras.

Problema da Árvore Geradora Mínima: Dado um grafo ponderado, o objetivo é encontrar uma árvore que conecta todos os vértices com o menor peso total. O algoritmo de Kruskal é um exemplo de algoritmo guloso que seleciona arestas de menor peso, evitando ciclos.

Vantagens dos Algoritmos Gulosos:

- Os algoritmos gulosos são geralmente mais simples de implementar e mais eficientes em termos de tempo de execução.
- São adequados para resolver problemas de otimização onde é possível fazer escolhas locais que levem a soluções globalmente ótimas.

Limitações dos Algoritmos Gulosos:

- Nem todos os problemas têm soluções ótimas usando abordagens gulosas. Algumas escolhas locais podem levar a soluções subótimas.
- A estratégia gulosa não é apropriada para problemas que envolvem dependências ou restrições mais complexas.

Decisão de Usar Algoritmos Gulosos:

- Decidir se um problema pode ser resolvido de maneira eficiente por meio de um algoritmo guloso requer análise cuidadosa das propriedades do problema.

Avaliação do Algoritmo:

- A eficácia de um algoritmo guloso pode ser avaliada comparando sua solução com a solução ótima, quando conhecida.
- Em muitos casos, mesmo que o algoritmo não encontre a solução global ótima, ele pode fornecer soluções aproximadas aceitáveis em tempo razoável.

Trabalho: 5pts

De acordo com o tema sorteado, escolha um algoritmo para aprofundar seus estudos.

- Algoritmos de Força Bruta
- Algoritmos de Divisão e Conquista
- Algoritmos Gulosos

Prepare uma apresentação (de 15 a 20min) para explicar características e a lógica do algoritmo escolhido, assim como sua análise quanto à complexidade e implementação.

Data: 06/11/2025

Algoritmos de Força bruta

- **Exaustividade;**
- **Simplicidade de Implementação;**
- **Alta Complexidade;**
- **Resultados Ótimos;**
- **Aplicação Limitada em Problemas Grandes;**

Vantagens	Desvantagens
Simplicidade de implementação. Apropriado para problemas pequenos ou bem definidos. Resultados precisos, pois considera todas as alternativas possíveis.	Ineficiente para grandes conjuntos de dados. Pode consumir muito tempo e memória. Muitas vezes impraticável para problemas em que o espaço de busca é grande.

Problemas Solucionáveis com Força Bruta

1.Problema da Mochila (Knapsack Problem):

$O(2^n)$

- Objetivo: Selecionar itens com o maior valor total que possam caber em uma mochila com capacidade limitada.
- Solução de Força Bruta: Testar todas as combinações possíveis de itens para encontrar o conjunto de maior valor.

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return (a > b) ? a : b;  
}
```

```
int knapsack(int W, int wt[], int val[], int n) {  
    if (n == 0 || W == 0)  
        return 0;  
    if (wt[n - 1] > W)  
        return knapsack(W, wt, val, n - 1);  
    else  
        return max(val[n - 1] + knapsack(W - wt[n - 1], wt, val, n - 1),  
                    knapsack(W, wt, val, n - 1));  
}
```

```
int main() {  
    int val[] = {60, 100, 120};  
    int wt[] = {10, 20, 30};  
    int W = 50;  
    int n = sizeof(val) / sizeof(val[0]);  
    printf("Valor máximo: %d\n", knapsack(W, wt, val, n));  
    return 0;  
}
```


2.Caixeiro Viajante (Traveling Salesman Problem - TSP): $O(n!)$

- Objetivo: Encontrar o caminho mais curto que passa por todas as cidades uma única vez e retorna à cidade de origem.
- Solução de Força Bruta: Gerar todas as permutações possíveis das rotas entre as cidades e selecionar a rota com menor distância total.
- Para o problema do Caixeiro Viajante com 5 cidades, um algoritmo de força bruta consideraria todas as permutações possíveis das 5 cidades (ou seja, $5! = 120$ rotas possíveis). Esse algoritmo compara a distância total de cada rota e escolhe a menor. Para um número maior de cidades, no entanto, a abordagem se torna inviável.

```
#include <stdio.h>
```

```
#define N 4
```

```
#define INF 999999
```

```
int tsp(int graph[][N], int v[], int pos, int n, int count, int cost, int *ans) {
    if (count == n && graph[pos][0]) {
        *ans = cost + graph[pos][0] < *ans ? cost + graph[pos][0] : *ans;
        return *ans;
    }

    for (int i = 0; i < n; i++) {
        if (!v[i] && graph[pos][i]) {
            v[i] = 1;
            tsp(graph, v, i, n, count + 1, cost + graph[pos][i], ans);
            v[i] = 0;
        }
    }
    return *ans;
}
```

```
int main() {
    int graph[N][N] = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    int v[N] = {0};
    v[0] = 1;
    int ans = INF;
    tsp(graph, v, 0, N, 1, 0, &ans);

    printf("Menor custo do Caixeiro Viajante: %d\n", ans);
    return 0;
}
```

3. Busca por Padrão (Pattern Matching): $O(n \cdot m)$

- Objetivo: Encontrar uma sequência específica em uma string, como uma palavra em um texto.
- Solução de Força Bruta: Comparar o padrão em cada posição da string para identificar se há uma correspondência.

```
#include <stdio.h>
#include <string.h>

void search(char* pat, char* txt) {
    int M = strlen(pat);
    int N = strlen(txt);

    for (int i = 0; i <= N - M; i++) {
        int j;
        for (j = 0; j < M; j++)
            if (txt[i + j] != pat[j])
                break;

        if (j == M)
            printf("Padrão encontrado na posição %d\n", i);
    }
}

int main() {
    char txt[] = "ABABDABACDABABCABAB";
    char pat[] = "ABABCABAB";
    search(pat, txt);
    return 0;
}
```

4.Problema das N-Rainhas (N-Queens Problem): $O(n!)$

- Objetivo: Colocar N rainhas em um tabuleiro N x N de forma que nenhuma delas se ataque.
- Solução de Força Bruta: Experimentar todas as combinações possíveis de posição para as rainhas e verificar as que atendem aos critérios.

```
bool solveNQUtil(int board[N][N], int col) {
    if (col >= N)
        return true;
    for (int i = 0; i < N; i++) {
        if (isSafe(board, i, col)) {
            board[i][col] = 1;

            if (solveNQUtil(board, col + 1))
                return true;

            board[i][col] = 0;
        }
    }
    return false;
}

printSolution(board);
return true;
}
```

```
#include <stdbool.h>
#include <stdio.h>
```

```
#define N 4
```

```
void printSolution(int board[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            printf(" %d ", board[i][j]);
        printf("\n");
    }
}
```

```
bool isSafe(int board[N][N], int row, int col) {
    for (int i = 0; i < col; i++)
        if (board[row][i])
            return false;
```

```
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;
```

```
    for (int i = row, j = col; j >= 0 && i < N; i++, j--)
        if (board[i][j])
            return false;
```

```
    return true;
}
```

```
bool solveNQ() {
    int board[N][N] = { {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0} };

    if (solveNQUtil(board, 0) == false) {
        printf("Solução não existe");
        return false;
    }
}
```

```
int main() {
    solveNQ();
    return 0;
}
```

5. Gerador de Subconjuntos

(Subset Sum Problem): $O(2^n)$

- Objetivo: Determinar se há um subconjunto de um conjunto de números que soma a um valor específico.
- Solução de Força Bruta: Gerar todos os subconjuntos possíveis e verificar se algum possui a soma desejada.

```
#include <stdio.h>
#include <stdbool.h>
```

```
bool isSubsetSum(int set[], int n, int sum) {
    if (sum == 0)
        return true;
    if (n == 0)
        return false;
    if (set[n - 1] > sum)
        return isSubsetSum(set, n - 1, sum);

    return isSubsetSum(set, n - 1, sum) || isSubsetSum(set, n - 1, sum - set[n - 1]);
}

int main() {
    int set[] = {3, 34, 4, 12, 5, 2};
    int sum = 9;
    int n = sizeof(set) / sizeof(set[0]);
    if (isSubsetSum(set, n, sum) == true)
        printf("Encontrado um subconjunto com a soma desejada");
    else
        printf("Nenhum subconjunto encontrado");
    return 0;
}
```

6. Permutações e Combinações: $O(n!)$

- Objetivo: Gerar todas as permutações ou combinações possíveis de uma lista de elementos.
- Solução de Força Bruta: Calcular todas as permutações ou combinações de elementos, que pode ser útil em problemas de análise de dados.

```
#include <stdio.h>
```

```
void swap(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void permute(int* arr, int start, int end) {  
    if (start == end) {  
        for (int i = 0; i <= end; i++)  
            printf("%d ", arr[i]);  
        printf("\n");  
    } else {  
        for (int i = start; i <= end; i++) {  
            swap(&arr[start], &arr[i]);  
            permute(arr, start + 1, end);  
            swap(&arr[start], &arr[i]);  
        }  
    }  
}
```

```
int main() {  
    int arr[] = {1, 2, 3};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    permute(arr, 0, n - 1);  
    return 0;  
}
```

7.Problema do Sudoku: $O(9^n)$

- Objetivo: Preencher um tabuleiro de Sudoku de acordo com as regras do jogo.
- Solução de Força Bruta: Testar todos os valores possíveis para cada célula vazia até encontrar uma configuração válida.

```
int main() {
    int grid[N][N] = {
        {5, 3, 0, 0, 7, 0, 0, 0, 0},
        {6, 0, 0, 1, 9, 5, 0, 0, 0},
        {0, 9, 8, 0, 0, 0, 0, 6, 0},
        {8, 0, 0, 0, 6, 0, 0, 0, 3},
        {4, 0, 0, 8, 0, 3, 0, 0, 1},
        {7, 0, 0, 0, 2, 0, 0, 0, 6},
        {0, 6, 0, 0, 0, 0, 2, 8, 0},
        {0, 0, 0, 4, 1, 9, 0, 0, 5},
        {0, 0, 0, 0, 8, 0, 0, 7, 9}
    };
    if (solveSudoku(grid, 0, 0))
        printGrid(grid);
    else
        printf("Solução não existe");
    return 0;
}
```

```
#include <stdio.h>
#include <stdbool.h>
```

```
#define N 9
```

```
bool isSafe(int grid[N][N], int row, int col, int num) {
    for (int x = 0; x < N; x++)
        if (grid[row][x] == num || grid[x][col] == num)
            return false;

    int startRow = row - row % 3, startCol = col - col % 3;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (grid[i + startRow][j + startCol] == num)
                return false;

    return true;
}

bool solveSudoku(int grid[N][N], int row, int col) {
    if (row == N - 1 && col == N)
        return true;
    if (col == N) {
        row++;
        col = 0;
    }
    if (grid[row][col] > 0)
        return solveSudoku(grid, row, col + 1);

    for (int num = 1; num <= N; num++) {
        if (isSafe(grid, row, col, num)) {
            grid[row][col] = num;
            if (solveSudoku(grid, row, col + 1))
                return true;
            grid[row][col] = 0;
        }
    }
    return false;
}

void printGrid(int grid[N][N]) {
    for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++)
            printf("%2d ", grid[row][col]);
        printf("\n");
    }
}
```

8. Palíndromo: $O(n)$

- Objetivo: Verificar se uma string é um palíndromo.
- Solução de Força Bruta: Comparar cada caractere da string com o seu oposto e verificar se todos coincidem.

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
```

```
bool isPalindrome(char str[]) {
    int l = 0;
    int h = strlen(str) - 1;

    while (h > l) {
        if (str[l++] != str[h--]) {
            return false;
        }
    }
    return true;
}
```

```
int main() {
    char str[] = "radar";
    if (isPalindrome(str))
        printf("A string é um palíndromo\n");
    else
        printf("A string não é um palíndromo\n");
    return 0;
}
```

9.Problema das Torres de Hanoi: $O(2^n)$

- Objetivo: Mover discos de uma torre para outra, seguindo regras específicas.
- Solução de Força Bruta: Resolver o problema de maneira recursiva, movendo cada disco de forma sequencial.

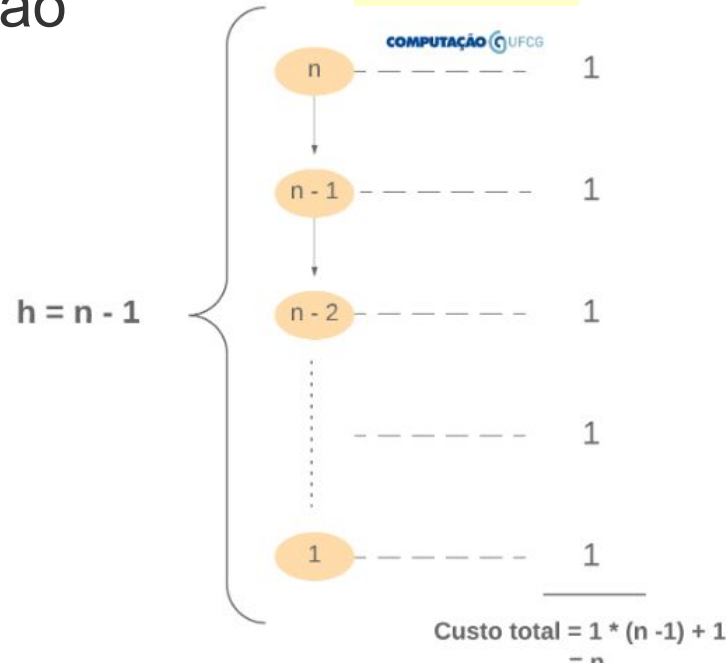
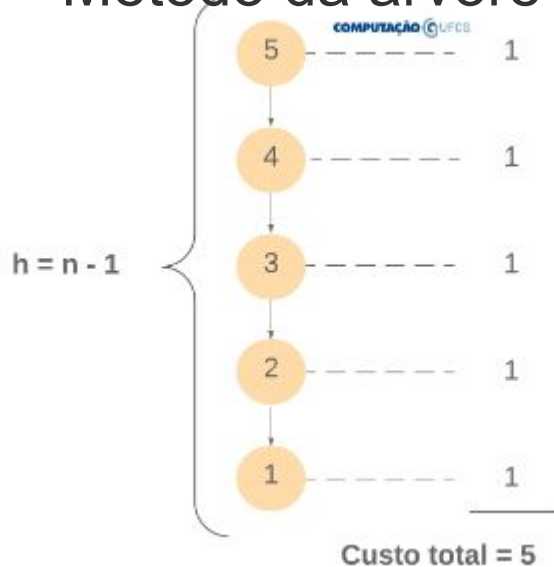
```
#include <stdio.h>
```

```
void hanoi(int n, char from_rod, char to_rod, char aux_rod) {  
    if (n == 1) {  
        printf("Mover disco 1 da haste %c para a haste %c\n", from_rod, to_rod);  
        return;  
    }  
    hanoi(n - 1, from_rod, aux_rod, to_rod);  
    printf("Mover disco %d da haste %c para a haste %c\n", n, from_rod, to_rod);  
    hanoi(n - 1, aux_rod, to_rod, from_rod);  
}  
  
int main() {  
    int n = 3;  
    hanoi(n, 'A', 'C', 'B');  
    return 0;  
}
```


Análise de algoritmos recursivos

Relação de Recorrência:

Método da árvore de recursão



$T(n) = a * T(n/b) + f(n)$, com $a \geq 1$, $b > 1$ e $f(n)$ não negativa.
dizemos que há **a** chamadas recursivas e que cada chamada recursiva divide a entrada em **b** partes. Além disso, a cada chamada recursiva, um custo $f(n)$ é adicionado.

Análise fatorial - recursivo

```
long fatorial(int n)
{
    if(n<=1) return 1;

    return n * fatorial(n-1);
}
```

Um exemplo clássico de algoritmo recursivo é o cálculo de fatorial:

$$0! = 1! = 1$$

$$n! = n \times (n - 1)!$$

Seja $T(n)$ a complexidade de tempo do fatorial recursivo:

Caso Base: $T(1) = a$;

Passo Recursivo: $T(n) = T(n - 1) + b$, $n > 1$.

$$T(2) = T(1) + b = a + b$$

$$T(3) = T(2) + b = a + 2b$$

$$T(4) = T(3) + b = a + 3b$$

Generalizando, temos que a forma fechada para esta recorrência é $T(n) = a + (n - 1)b$.

Logo, sendo a e b constantes, $T(n) = O(n)$.

Relação recorrência merge

```
public void mergeSort(int[] v, int ini, int fim)
{
```

```
    If (ini < fim) {
```

```
        int meio = (ini + fim) / 2;
```

```
        mergeSort(v, ini, meio);
```

```
        mergeSort(v, meio + 1, fim);
```

```
        merge(v, ini, meio, fim);
```

```
    }
```

```
}
```

1op

3 op

$t(n/2)$

$t(n/2)$

$O(n)$

$T_n = O(1) + t(n/2) + t(n/2) + O(n)$

$T_n = 2t(n/2) + n$

$h = \log n$
 $n/2^0$
 $n/2^1$
 $n/2^2$
 $n/2^{**h}$

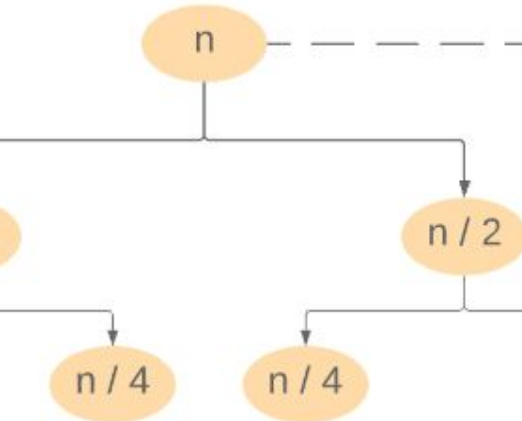
$$n/2^h = 1$$

$$2^h = n$$

$$\log 2^h = \log n$$

$$h \log 2 = \log n$$

$$h = \log n$$


 n
 $2 * n/2$
 $4 * n/4$

Custo total = $n * \log n$

Teorema mestre

Ex1: Para a relação de recorrência $T(n)=8*T(n/2)+1000*n^2$

- $a=8$
- $b=2$
- $f(n)=1000*n^2$

Comparando: temos que $1000*n^2 < n^3$

Portanto, $T(n)=\Theta(n^3)$

Se $f(n) < n^{\log_b a}$, então $T(n) = \Theta(n^{\log_b a})$.

Se $f(n) = n^{\log_b a}$, então $T(n) = \Theta(f(n) * \log_b n)$.

Se $f(n) > n^{\log_b a}$, então $T(n) = \Theta(f(n))$.

Teorema mestre

Ex2: $T(n)=2*T(n/2)+10*n$

- $a=2$
- $b=2$
- $f(n)=10*n$

Se $f(n) < n^{**} \log_b^a$, então $T(n) = \Theta(n^{**} \log_b^a)$.

Se $f(n) = n^{**} \log_b^a$, então $T(n) = \Theta(f(n) * \log_b^n)$.

Se $f(n) > n^{**} \log_b^a$, então $T(n) = \Theta(f(n))$.

Comparando: $10*n == n$, pois comparamos a ordem de grandeza das funções e, quando fazemos isso, as constantes não importam. Portanto, aplicando a segunda regra do Teorema Mestre, podemos afirmar que

$$T(n) = \Theta(n * \log_2 n)$$

Teorema mestre

Ex3: $T(n)=2*T(n/2)+n^2$, temos:

- $a=2$
- $b=2$
- $f(n)=n^2$

Comparando: $n^2 > n$.

Portanto: $T(n)=\Theta(n^2)$.

Se $f(n) < n^{**} \log_b^a$, então $T(n) = \Theta(n^{**} \log_b^a)$.

Se $f(n) = n^{**} \log_b^a$, então $T(n) = \Theta(f(n) * \log_b^n)$.

Se $f(n) > n^{**} \log_b^a$, então $T(n) = \Theta(f(n))$.